

JAYPEE INSTITUTE OF INFORMATION TECHNOLOGY

DATA STRUCTURES PBL REPORT



SoundWave: A Dynamic Music Streaming Platform

Submitted by:

Rohan Lal (B12) : 2401030108

Swastik Sangwan (B12) : 2401030098

Neal Pandita (B12) : 2401030110

BTECH SEM III

Submitted To:

Dr. Varun Srivastava

Ms. Anupama Padha

Mr. Rajiv Kumar Mishra

TEAM CONTRIBUTION

1. Swastik – Algorithms & Data Structures

Implemented the efficient **Trie** data structure for search functionality and the **MaxHeap** architecture for the recommendation engine in **search.js** and **heap.js**.

2. Neal – State Management & Player Logic

Developed **storage.js** for persistent **localStorage** handling and **player.js** to simulate playback timing, queuing, and history tracking.

3. Rohan – UI Architecture & DOM Manipulation Designed the **index.html** layout and **ui.js** module to handle event listeners, dynamic rendering of song lists, and playlist management interactions.

ABSTRACT

SoundWave is a robust, modular Single-Page Application (SPA) designed for modern web browsers. It automates the simulation of a digital music player, utilizing advanced data structures to optimize search and discovery features without a backend database.

The project is developed using:

- **Vanilla JavaScript (ES6+)** – Core logic, modules, and DOM manipulation.
- **Local Storage API** – Data persistence and state management.
- **Trie & Heaps** – Optimized search indexing and algorithmic recommendations.

This project demonstrates advanced Computer Science concepts such as Client-Side Rendering, Algorithm Efficiency ($O(L)$ search), State Persistence, and modular software architecture.

INTRODUCTION

In modern web development, managing client-side state efficiently is critical. While many music players rely heavily on server-side processing, they often suffer from latency during search and interaction.

SoundWave provides:

- **Real-time Search:** Instant autocomplete using a Prefix Tree (Trie).
- **Smart Recommendations:** A recommendation algorithm using Max-Heaps to score songs based on tag/artist similarity.
- **State Persistence:** Automatic saving of playlists, history, and queue via `storage.js`.
- **Playback Simulation:** A virtual audio engine that mimics track progress and queue advancement.
- **Seamless Integration:** Modular architecture allowing easy extension of features.

This project emphasizes how Data Structures (Tries/Heaps) act as a powerful engine for frontend applications, enhancing user experience significantly.

OBJECTIVES

The primary objectives of this project are:

- To design a non-intrusive, aesthetically pleasing Single-Page Application.
- To implement automated playlist management (Create/Delete/Rename) using modular JavaScript.
- To utilize **Trie Data Structures** for efficient $O(L)$ search instead of resource-heavy array filters ($O(N)$).
- To create a robust state management system that handles page reloads without data loss.
- To maintain detailed history logs of user listening habits for generating recommendations.

PROBLEM STATEMENT

Standard client-side music players often fail to scale efficiently when searching through large datasets and lack intelligent content discovery features. Manual searching for songs by exact title is tedious and prone to user error.

- **Current Issue:** Linear search methods ($O(N)$) cause lag in large libraries; static playlists do not adapt to user preferences.
- **Solution:** A "smart" client-side utility that enforces efficient indexing (Trie) and automated curation (Heap) based on listening history.

SCOPE OF PROJECT

In Scope:

- **Song Library Management:** JSON-based seeding and management of song metadata.
- **Algorithmic Search:** Prefix-based search suggestions using Tries.
- **User Customization:** Creation of custom playlists and "Liked Songs" management.
- **Persistence:** `localStorage` integration to save state across sessions.
- **Recommendation Engine:** Weighted scoring system based on tags, artists, and albums.

Out of Scope:

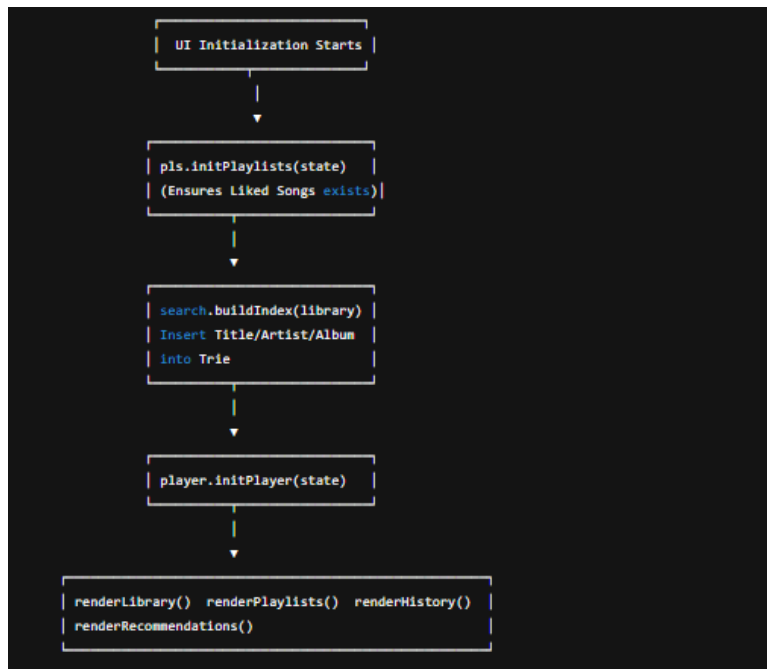
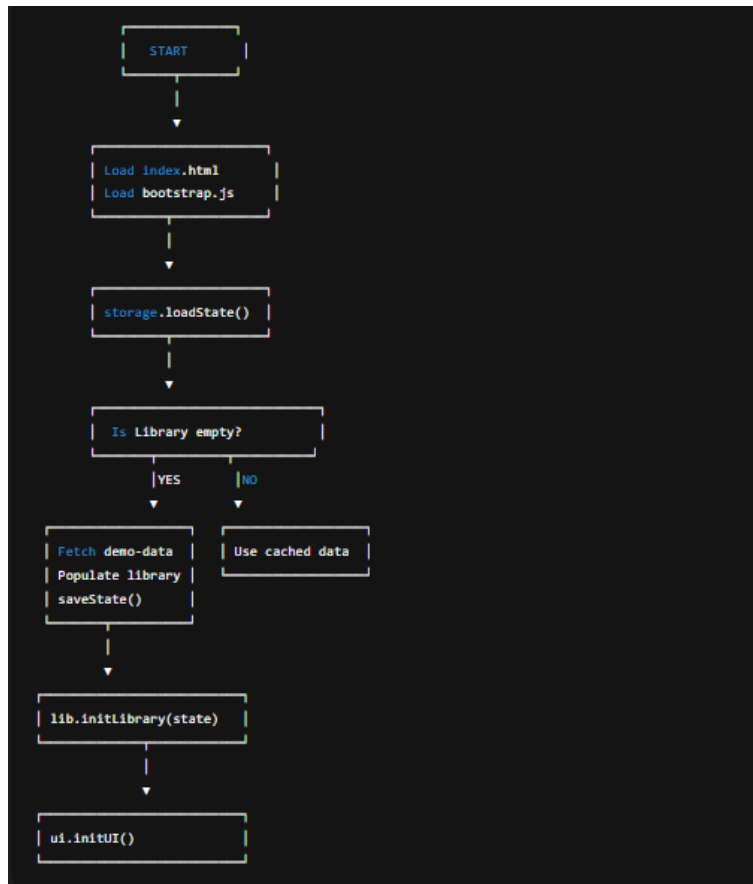
- **Audio Decoding:** Real MP3/FLAC file processing (Simulation only).
- **Cloud Backend:** No server-side database (Firestore/SQL).
- **Mobile App:** Native mobile wrapper (Web-only)

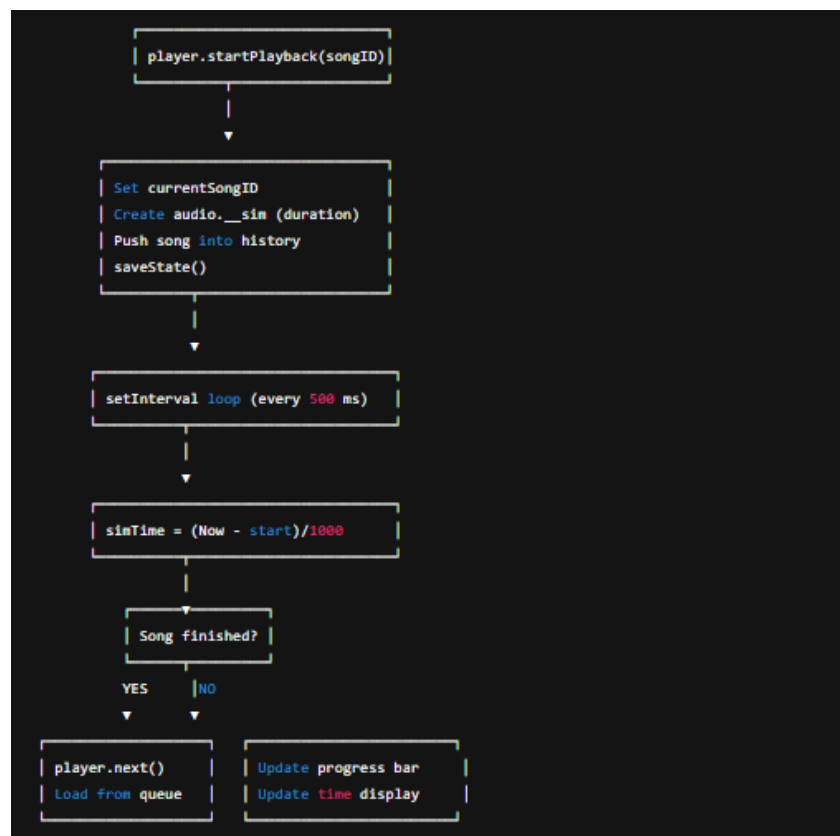
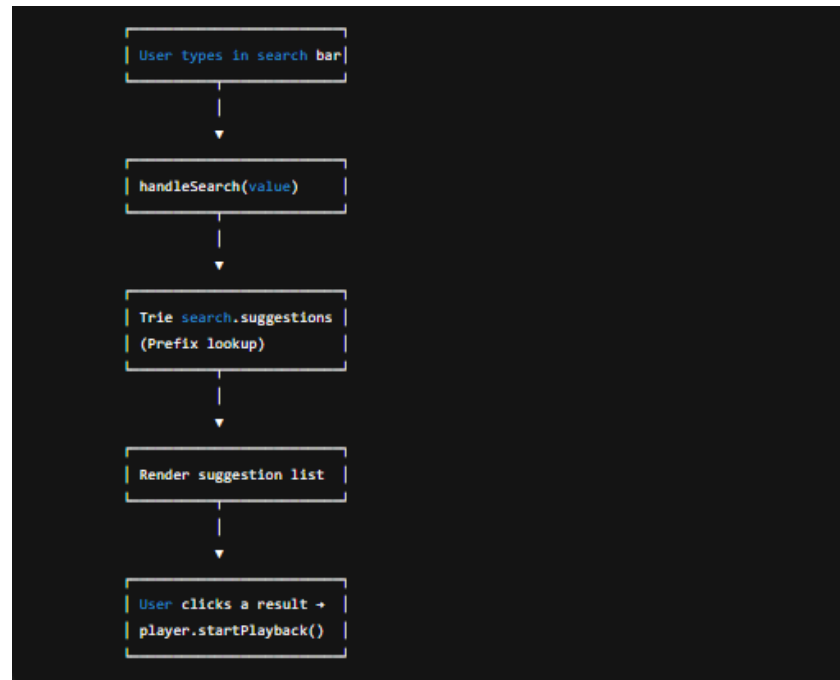
SYSTEM ARCHITECTURE

The system follows a modular architecture typical of modern SPAs:

1. **User Interface Layer (View):** The user interacts via the DOM (`index.html`, `styles.css`) triggered by `ui.js`.
2. **Logic Layer (Controller):**
 - `player.js`: The brain of the playback simulation.
 - `search.js`: The algorithmic engine handling Tries and Heaps.
3. **System Layer (Model/Storage):**
 - `storage.js`: Handles read/write operations to the browser's `LocalStorage`.
 - `library.js`: Manages the in-memory state of the song database.

CODEFLOW







FILE SYSTEM & CONFIGURATION DESIGN

Instead of a traditional database, this project uses **JSON** and **LocalStorage** to store state and logs, adhering to the concept that the client maintains the session.

Configuration/Data File (`demo-data.json`): Stores the initial seed library.

JSON

- {
- "library": [- {"id": "s1", "title": "Blinding Lights", "artist": "The Weeknd", ...},
- ...
-]
- }

State Object (Managed by `storage.js`): A persistent record of the application status.

- **Format:** JSON string saved under key `soundwave_state_v2`.
- **Modules:**
 - `js/ds/`: Directory containing Data Structures (`trie.js`, `heap.js`).
 - `js/bootstrap.js`: Entry point responsible for initialization.

CODE SNIPPETS

1. Trie Implementation (Search Logic) This snippet from `trie.js` demonstrates the insertion logic for the prefix tree.

JavaScript

```
export class Trie {
  constructor(){ this.root = new TrieNode(); }
  insert(word, id){
    if(!word) return;
    let node = this.root;
    for(const ch of word){
      if(!node.children[ch]) node.children[ch] = new
TrieNode();
      node = node.children[ch];
      node.ids.add(id);
    }
    node.end = true;
  }
}
```

2. Core Logic (Recommendation Engine) This function in `search.js` shows how Max-Heaps are used to calculate the top 5 recommended songs based on listening history.

JavaScript

```
export function recommend(library, history, limit = 5){
  const heap = new MaxHeap((a,b)=> a.score - b.score);
  // ... history processing ...
}
```

```

Object.entries(scored).forEach(([id, score])=>{
  heap.push({ id, score });
});

const res = [];
while(res.length < limit && heap.size() > 0){
  res.push(heap.pop().id); // Extract max score
}
return res;
}

```

3. State Persistence (Storage) The `storage.js` script safely persists the state tree to the browser.

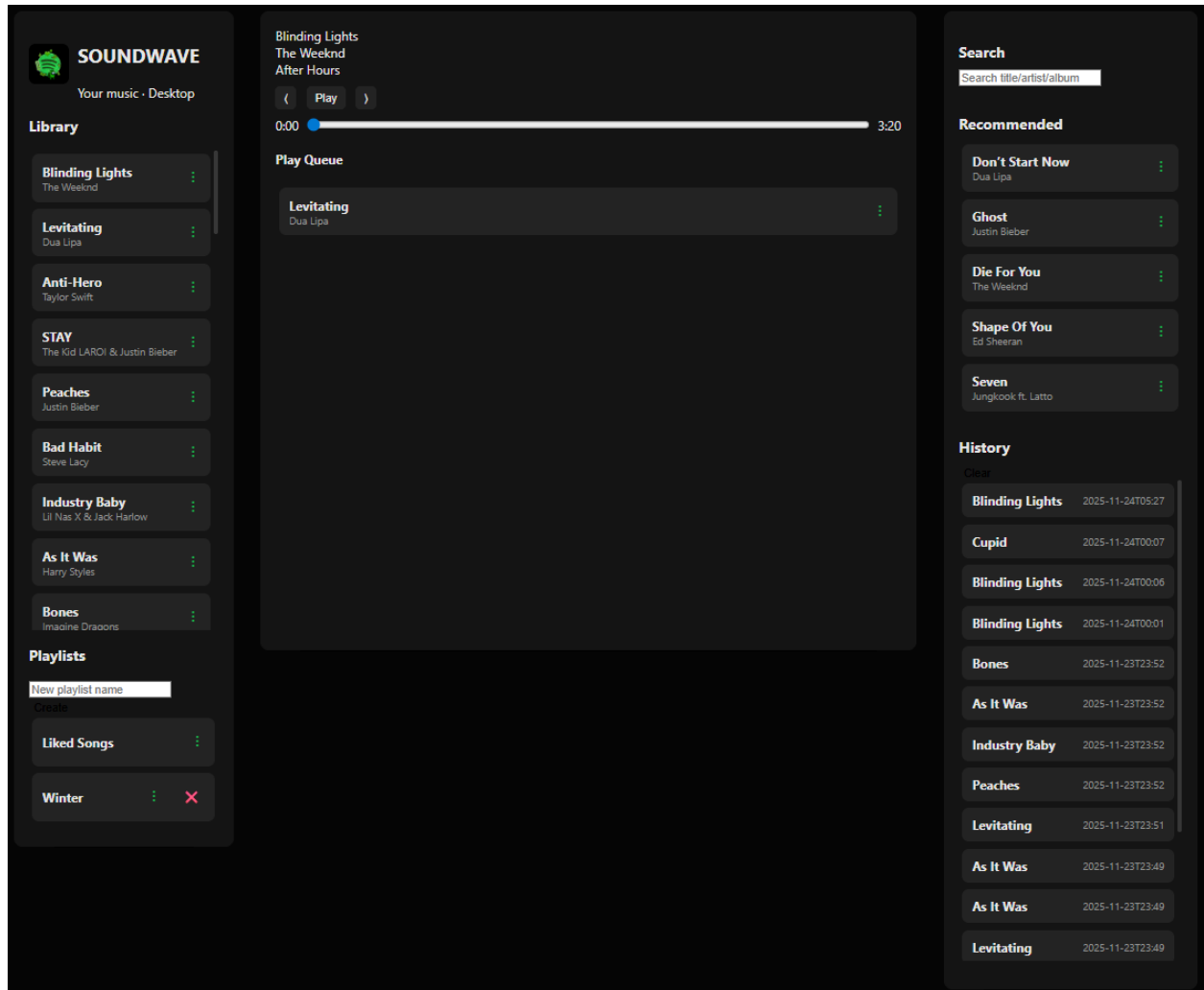
JavaScript

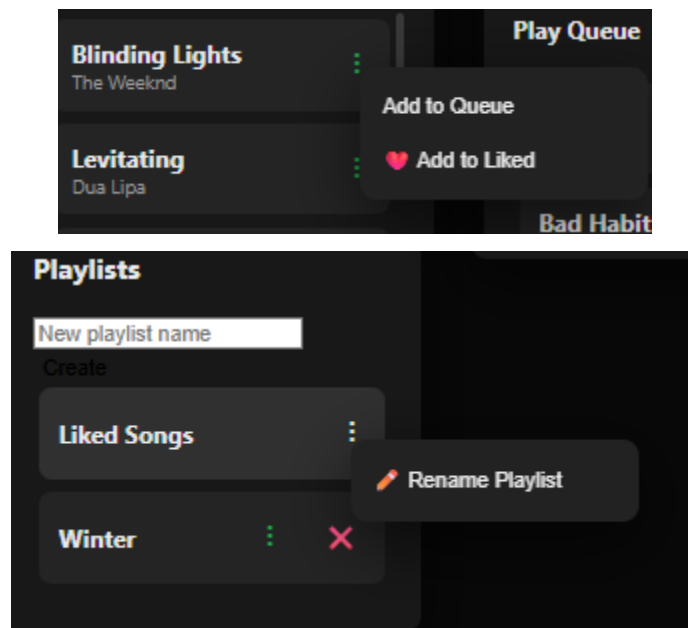
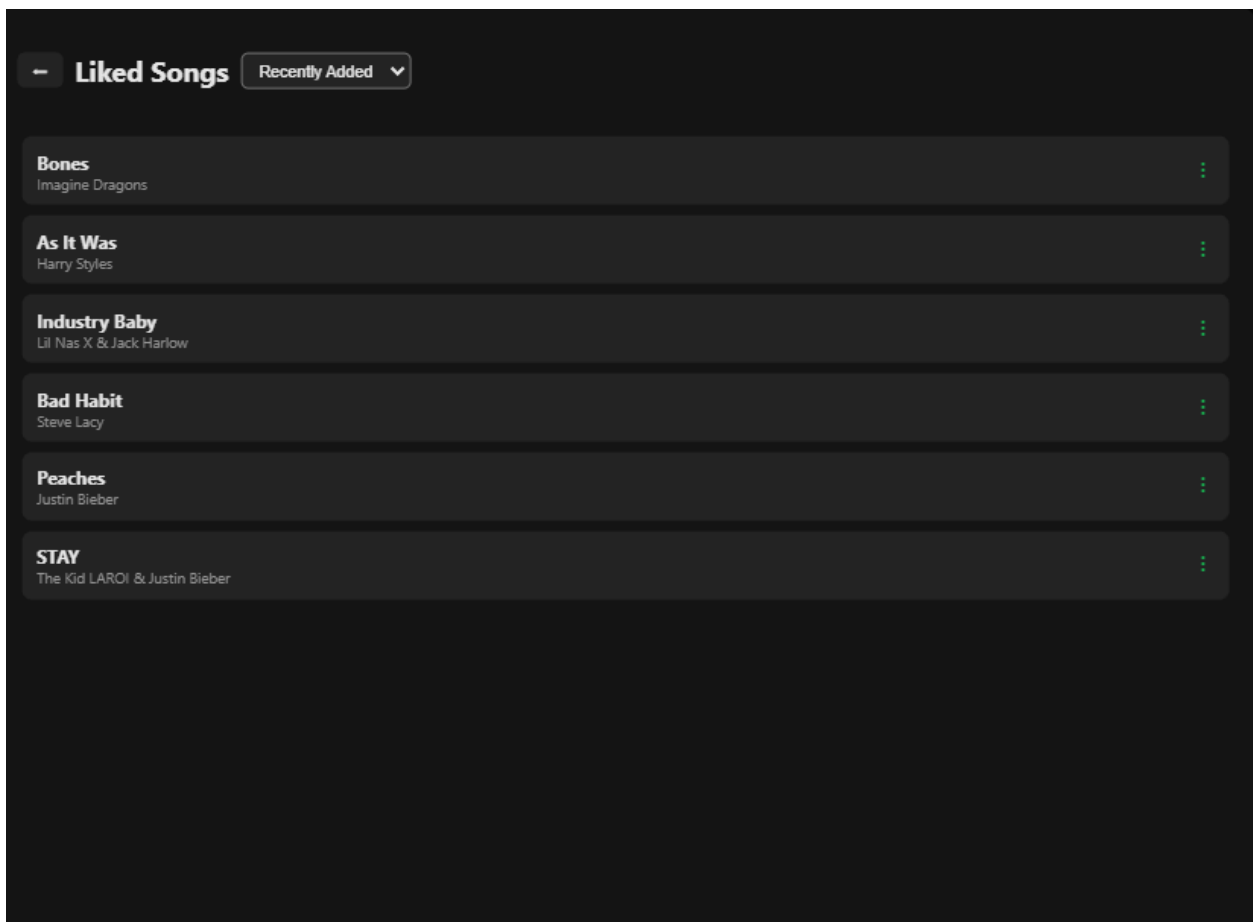
```

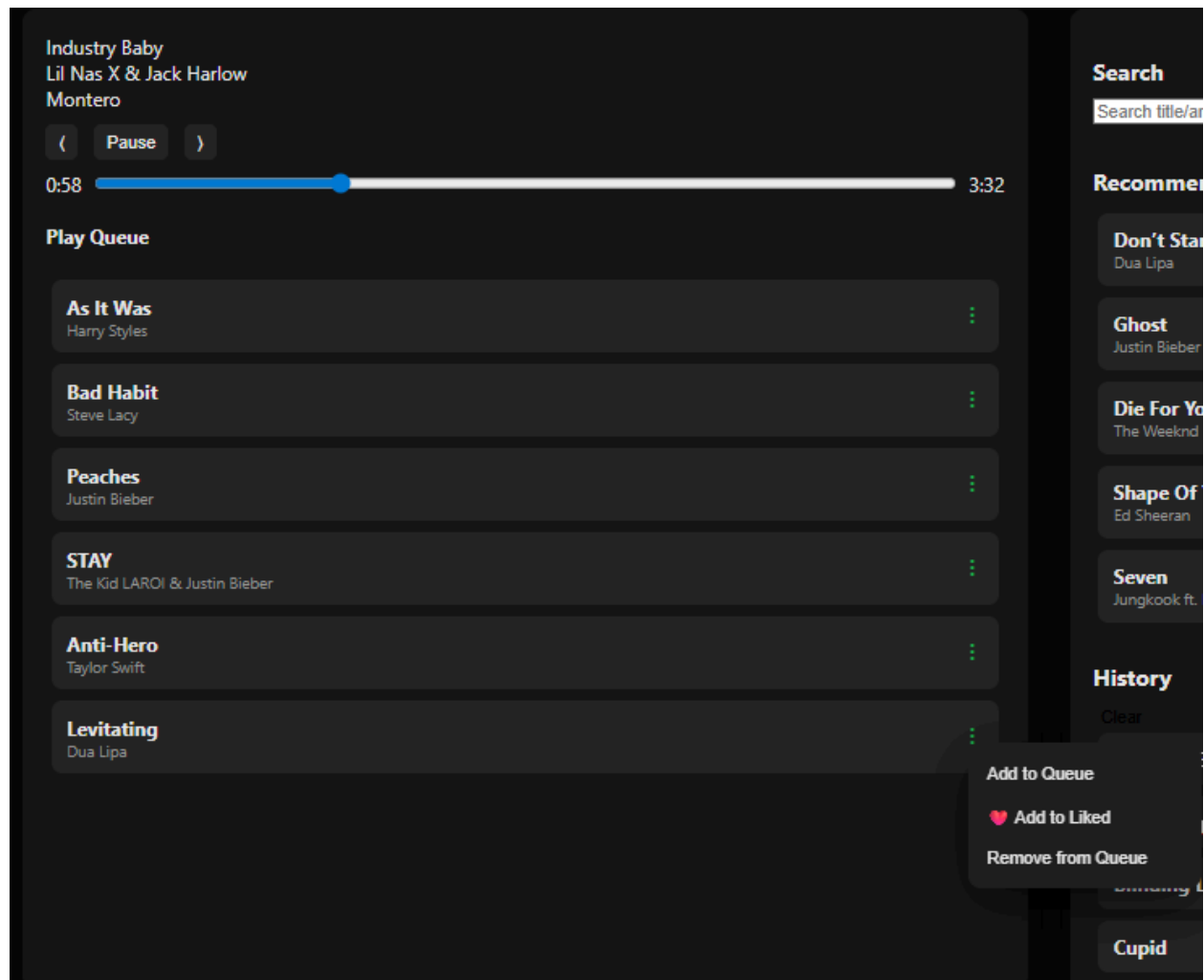
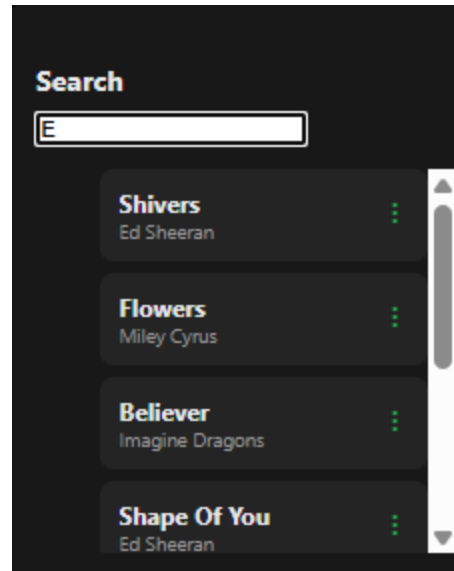
export function saveState(state) {
  try {
    const safe = {
      library: state.library || [],
      playlists: state.playlists || {},
      history: state.history || [],
      queue: state.queue || []
    };
    localStorage.setItem(KEY, JSON.stringify(safe));
  } catch (err) {
    console.error("State save failed:", err);
  }
}

```

OUTPUT SNIPPETS







TESTING & VALIDATION

The project includes a robust set of logical tests to validate the environment. Below are the test cases executed.

The following table:

Test Case ID	Test Description	Action / Query Executed	Expected Result	Status
TC-01	App Initialization	Load <code>index.html</code>	Library renders 50 songs; Console shows "UI Initialized".	 Pass
TC-02	Search Functionality	Type "wee" in search bar	Suggests "The Weeknd" via Trie traversal.	 Pass
TC-03	Playback Simulation	Click "Play" on a track	Progress bar advances; Queue updates.	 Pass
TC-04	Playlist Creation	Click "Create", Enter Name	New playlist appears in sidebar; Persists on refresh.	 Pass

TC-05	Recommendation Logic	Play 3 Pop songs	"Recommended" list updates with similar Pop tracks.	 Pass
TC-06	History Logging	Play a song to completion	Song ID appears in History list.	 Pass
TC-07	Data Persistence	Refresh Browser (F5)	Current song, queue, and playlists are restored.	 Pass

The testing suite confirms that the system is stable and interacts correctly with the underlying browser storage.

CONCLUSION

The **SoundWave** project successfully demonstrates the power of Vanilla JavaScript and Data Structures for building complex frontend applications.

63

Key Learning Outcomes:

- **Optimization:** We replaced linear array searches with a **Trie**, reducing search complexity from $O(N)$ to $O(L)$.
- **System Integration:** By using ES6 Modules (`import/export`), we learned how to architect maintainable codebases, separating concerns between UI, Data, and Logic.
- **Resilience:** The application employs defensive programming (checking `!state` or `!library`) and fallback defaults to ensure it runs even if `localStorage` is corrupted.
- **User Experience:** Through the use of a dark-mode CSS Grid layout and instant interactions, the tool provides a professional user experience similar to standard streaming apps.

This project serves as a practical foundation for understanding Frontend Engineering, State Management, and the application of Algorithmic Theory.

REFERENCES

- **MDN Web Docs** – JavaScript & DOM Manipulation
- **Introduction to Algorithms (CLRS)** – Heaps & Tries
- **SoundWave Source Code** – [search.js](#), [trie.js](#), [player.js](#)